

Programación de Sistemas

Unidad 2: Análisis léxico

Nota de clase 04: Expresiones regulares y autómatas finitos

Manuel Soto Romero

Araceli Liliana Reyes Cabello

Semestre 2026-2

Colegio de Ciencia y Tecnología UACM

En la nota anterior distinguimos token, lexema y patrón. El patrón describía, todavía en lenguaje natural, la forma de los lexemas asociados con una categoría. Para construir un analizador léxico necesitamos dar el siguiente paso: escribir esas descripciones con precisión y contar con un procedimiento que decida si una cadena las satisface.

En esta nota utilizaremos expresiones regulares para especificar patrones y autómatas finitos para reconocerlos. Primero reconstruiremos los conceptos indispensables de lenguajes formales; después compararemos autómatas finitos no deterministas y deterministas; finalmente, seguiremos la ruta de conversión y minimización que conecta ambas herramientas. Los ejemplos se aplicarán a los tokens del IMP básico. La coordinación de todos los patrones y su implementación con LARK se desarrollarán en el tema siguiente.

Ruta de lectura (35–40 minutos)

Lectura esencial: sigue el caso inicial; identifica qué lenguaje denota cada expresión regular; compara AFN y AFD; y revisa el ejemplo completo $a | b$ en la ruta expresión regular $\rightarrow$ AFN $\rightarrow$ AFD $\rightarrow$ AFD mínimo. Termina cotejando la salida conceptual del fragmento de IMP.

Lectura complementaria: revisa las definiciones formales de alfabeto, cadena y lenguaje; las plantillas de Thompson; el detalle de la cerradura- ϵ ; y el refinamiento de particiones. Estas partes sirven como nivelación y pueden retomarse cuando se necesiten.

Caso inicial. Dos formas de describir un patrón

Consideremos el siguiente fragmento de IMP:

```
while x2 <= 20 do x2 := x2 + 1
```

En la nota anterior podíamos describir `x2` como “una o más letras seguidas de cero o más dígitos”. En esta nota escribiremos esa descripción mediante una expresión regular:

$$LOC = letra^+ digito^*.$$

Esta expresión especifica un conjunto de cadenas, pero todavía necesitamos un mecanismo que reciba una cadena como `x2` y responda si pertenece a ese conjunto. Un autómata finito realizará ese reconocimiento.

1. Del patrón a un reconocedor

Especificar y reconocer son tareas relacionadas, pero distintas. Una **especificación** describe cuáles cadenas pertenecen a una categoría léxica; un **reconocedor** recibe una cadena y decide si pertenece al lenguaje

especificado. En el análisis léxico utilizaremos expresiones regulares para la primera tarea y autómatas finitos para la segunda.

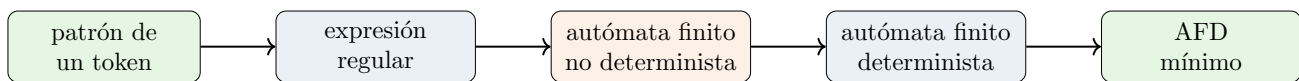
Antes de formalizar esta relación necesitamos tres conceptos básicos.

Definición 1. (Alfabeto, cadena y lenguaje)

Un **alfabeto** Σ es un conjunto finito de símbolos. Una **cadena** sobre Σ es una secuencia finita de símbolos de ese alfabeto. La cadena sin símbolos se llama **cadena vacía** y se representa con ε . Un **lenguaje** sobre Σ es un conjunto de cadenas formadas con símbolos de Σ .

En nuestro caso, los símbolos del alfabeto son caracteres del programa fuente. El lenguaje asociado con NUM, por ejemplo, contiene las cadenas formadas por uno o más dígitos decimales. 0, 7 y 205 pertenecen a ese lenguaje; la cadena vacía y 2x no pertenecen.

La ruta conceptual que seguiremos puede resumirse así:



Las conversiones conservan el lenguaje: cambian la representación, pero no el conjunto de cadenas aceptadas. Esta propiedad permite elegir una notación cómoda para especificar y una representación eficiente para reconocer.

Observación 1. Un lenguaje no es una cadena

La expresión regular de NUM describe un conjunto potencialmente infinito de lexemas. 205 es una cadena concreta de ese conjunto; no es la expresión regular ni el lenguaje completo. Esta distinción continúa la separación entre patrón y lexema de la nota anterior.

2. Expresiones regulares y definición de tokens

Una expresión regular se construye a partir de símbolos y operaciones. A cada expresión r le corresponde un lenguaje, que representaremos con $L(r)$.

Definición 2. (Expresión regular)

Sea Σ un alfabeto. La cadena vacía ε es una expresión regular y denota el lenguaje $\{\varepsilon\}$. Cada símbolo $a \in \Sigma$ es una expresión regular y denota $\{a\}$. Si r y s son expresiones regulares, también lo son $r \mid s$, rs , r^* y (r) .

Las tres operaciones principales tienen el siguiente significado:

Operación	Notación	Lenguaje resultante
Unión	$r \mid s$	Las cadenas que pertenecen a $L(r)$, a $L(s)$ o a ambos.
Concatenación	rs	Las cadenas formadas al concatenar una cadena de $L(r)$ con una de $L(s)$.
Cerradura de Kleene	r^*	Las cadenas formadas al concatenar cero o más cadenas de $L(r)$; incluye ε .

Por convención, la cerradura tiene mayor precedencia que la concatenación, y la concatenación tiene mayor precedencia que la unión. Así, $a \mid b^*c$ se interpreta como $a \mid ((b^*)c)$.

En la especificación de tokens también se utilizan abreviaturas. La cerradura positiva r^+ significa “una o más repeticiones” y equivale a rr^* . Una clase como $[0-9]$ abrevia la unión de los diez dígitos. Estas extensiones hacen más breve la escritura, pero no aumentan los lenguajes que pueden describirse.

Ejemplo resuelto. Qué cadenas denota cada expresión

Sea $\Sigma = \{a, b\}$.

Expresión	Lenguaje denotado
$a \mid b$	$\{a, b\}$.
ab	$\{ab\}$.
a^*	$\{\varepsilon, a, aa, aaa, \dots\}$.
$(a \mid b)^*$	Todas las cadenas finitas formadas con a y b , incluida ε .
$a(a \mid b)^*$	Las cadenas no vacías sobre $\{a, b\}$ cuyo primer símbolo es a .

Para evitar repetir subexpresiones podemos asignarles nombres y construir **definiciones regulares**. La especificación léxica aprobada para las dos categorías variables del IMP básico es:

$$\begin{aligned}
 \text{letra} &= [\text{A-Za-z}], \\
 \text{digito} &= [0-9], \\
 \text{LOC} &= \text{letra}^+ \text{digito}^*, \\
 \text{NUM} &= \text{digito}^+.
 \end{aligned}$$

La definición de LOC tiene dos partes. letra^+ exige una o más letras; después, digito^* admite cero o más dígitos. Por ello, los dígitos pueden aparecer al final, pero no al inicio ni entre dos grupos de letras.

Token	Patrón	Sí satisface	No satisface
LOC	$\text{letra}^+ \text{digito}^*$	x, total, x20	20x, x2y, x_1
NUM	digito^+	0, 25, 2048	ε , 2x, -5

El signo de -5 no forma parte de NUM: en IMP, - corresponde al token MINUS y 5 corresponde a NUM. La estructura sintáctica determinará después cómo se agrupan.

Las palabras reservadas, operadores y delimitadores del IMP básico tienen patrones fijos. Algunos ejemplos son:

Token	Lexema fijo	Token	Lexema fijo
WHILE	while	DO	do
ASSIGN	:=	LE	<=
PLUS	+	SEMI	;
LPAR	(	RPAR	)

Observación 2. Los patrones pueden coincidir

La cadena `while` satisface el patrón de `L0C`, pero también es el lexema fijo de `WHILE`. Las expresiones regulares especifican ambos casos; todavía falta establecer cómo coordinar las reglas para elegir un token. Esa decisión pertenece al diseño del analizador léxico del tema 2.3.

Mini-checkpoint. Lee la expresión antes de probar cadenas

Considera $\text{letra}^+ \text{digito}^*$. Clasifica `abc`, `abc12`, `ab12c` y `12`.

Resultado esperado: `abc` y `abc12` satisfacen la expresión. `ab12c` no la satisface porque aparece una letra después de comenzar los dígitos; `12` tampoco, porque falta la parte inicial de una o más letras.

3. Autómatas finitos deterministas y no deterministas

Una expresión regular dice *qué* cadenas forman un lenguaje. Un autómata finito permite decidir si una cadena concreta pertenece a él. Su memoria está representada por un conjunto finito de estados y cambia conforme consume los símbolos de la entrada.

Definición 3. (Autómata finito no determinista)

Un **autómata finito no determinista** (AFN) es una quintupla $N = (Q, \Sigma, \delta, q_0, F)$, donde Q es un conjunto finito de estados, Σ es el alfabeto, $q_0 \in Q$ es el estado inicial, $F \subseteq Q$ es el conjunto de estados de aceptación y

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \longrightarrow 2^Q$$

indica los posibles estados siguientes.

En un AFN puede haber varias transiciones con el mismo símbolo y también transiciones con ε , que cambian de estado sin consumir un símbolo. Una cadena se acepta si **existe al menos un recorrido** que consume toda la entrada y termina en un estado de aceptación.

Definición 4. (Autómata finito determinista)

Un **autómata finito determinista** (AFD) es una quintupla $D = (Q, \Sigma, \delta, q_0, F)$ cuya función de transición es

$$\delta : Q \times \Sigma \longrightarrow Q.$$

Para cada estado y cada símbolo del alfabeto existe exactamente un estado siguiente.

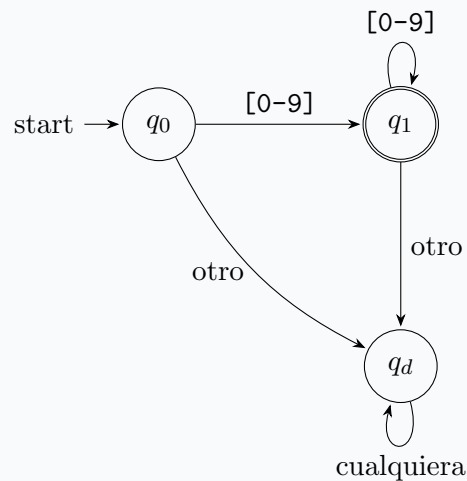
En un AFD no hay transiciones con ε ni alternativas para el mismo par estado-símbolo. Por ello, una cadena determina un único recorrido.

Aspecto	AFN	AFD
Siguiente estado	Puede ser un conjunto de estados.	Es un solo estado.
Transiciones ε	Se permiten.	No se permiten.
Aceptación	Basta un recorrido de aceptación.	Se sigue el único recorrido posible.
Uso en la ruta de construcción	Facilita componer subexpresiones.	Facilita reconocer la entrada mediante una transición por símbolo.

Aunque su funcionamiento es distinto, AFN y AFD tienen el mismo poder expresivo: para cada AFN existe un AFD que reconoce el mismo lenguaje.

Ejemplo resuelto. Un AFD para NUM

La expresión digito^+ requiere consumir al menos un dígito. El siguiente AFD reconoce una cadena completa que satisface ese patrón. El doble círculo señala el estado de aceptación.



El estado q_0 representa que todavía no se ha leído un dígito. q_1 recuerda que ya se consumió al menos uno y permite consumir más. q_d es un estado muerto: una vez alcanzado, la cadena completa ya no puede satisfacer el patrón.

Entrada	Recorrido	Resultado
25	$q_0 \xrightarrow{2} q_1 \xrightarrow{5} q_1$	Acepta.
ε	Permanece en q_0 .	Rechaza.
2x	$q_0 \xrightarrow{2} q_1 \xrightarrow{x} q_d$	Rechaza.

Aquí comprobamos cadenas completas contra un patrón. Un lexer real debe encontrar un prefijo que forme un lexema y dejar el carácter siguiente para continuar el análisis. Esa coordinación se estudiará al diseñar e implementar el analizador léxico.

4. De expresiones regulares a autómatas

La construcción presentada por Aho, Lam, Sethi y Ullman sigue dos conversiones. Primero aplica el método de McNaughton–Yamada–Thompson para obtener un AFN a partir de una expresión regular. Después aplica la construcción de subconjuntos para convertir ese AFN en un AFD equivalente.

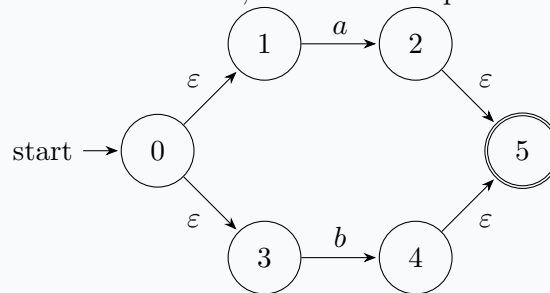
4.1 De una expresión regular a un AFN

El método de Thompson analiza la expresión por subexpresiones y construye para cada una un AFN con un estado inicial y uno de aceptación. Sus plantillas siguen la definición recursiva de las expresiones regulares:

- ★ un símbolo a se representa con una transición etiquetada con a ;
- ★ ε se representa con una transición ε ;
- ★ la unión agrega un nuevo inicio que puede entrar mediante ε a cualquiera de los dos AFN y un nuevo estado de aceptación común;
- ★ la concatenación enlaza la salida del primer AFN con la entrada del segundo;
- ★ la cerradura agrega recorridos ε para omitir la subexpresión o repetirla.

Ejemplo resuelto. AFN de Thompson para $a \mid b$

La unión permite elegir, sin consumir entrada, entre el AFN que reconoce a y el que reconoce b .



La entrada a puede seguir el recorrido $0 \xrightarrow{\varepsilon} 1 \xrightarrow{a} 2 \xrightarrow{\varepsilon} 5$. La entrada b sigue la rama inferior. Ambas terminan en el estado de aceptación; ab no puede consumir por completo la entrada y se rechaza.

4.2 De un AFN a un AFD: construcción de subconjuntos

Un estado del AFD representará el conjunto de estados en los que el AFN podría encontrarse después de leer la misma entrada. Para calcularlo utilizamos dos operaciones.

Definición 5. (Cerradura- ε y movimiento)

La **cerradura- ε** de un conjunto T contiene los estados de T y todos los estados alcanzables desde ellos usando únicamente transiciones ε . $\text{mover}(T, a)$ contiene los estados alcanzables desde algún estado de T mediante una transición etiquetada con a .

Como cada estado del AFD es un subconjunto de Q , los estados posibles pertenecen al **conjunto potencia** $\mathcal{P}(Q)$. Si el AFN tiene n estados, existen hasta 2^n subconjuntos, aunque normalmente sólo algunos son alcanzables desde el estado inicial. En lugar de escribirlos todos y eliminar después los inalcanzables, podemos construir únicamente los que se descubren mediante los siguientes pasos:

1. Calcula $A_0 = \text{cerradura}_\varepsilon(\{q_0\})$. Éste será el estado inicial del AFD y el primer conjunto pendiente de procesar.
2. Para cada conjunto pendiente T y cada símbolo $a \in \Sigma$, calcula

$$U = \text{cerradura}_\varepsilon(\text{mover}(T, a)).$$

3. Si U todavía no aparece, agrégalo como estado pendiente; registra además la transición $T \xrightarrow{a} U$.
4. Marca U como estado de aceptación si contiene algún estado de aceptación del AFN, es decir, si $U \cap F \neq \emptyset$.
5. Repite el proceso hasta que no queden conjuntos pendientes. Después pueden asignarse nombres breves a los subconjuntos alcanzables.

Para el AFN de $a \mid b$, el estado inicial del AFD es

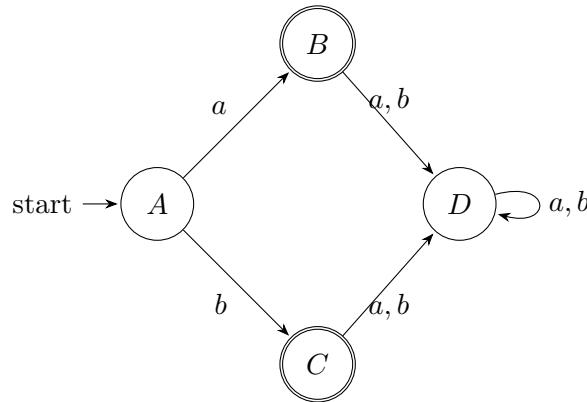
$$A = \text{cerradura}_\epsilon(\{0\}) = \{0, 1, 3\}.$$

Desde A , leer a permite llegar primero a 2; su cerradura también contiene 5. De manera análoga, leer b lleva primero a 4 y después a 5:

$$\begin{aligned} \text{mover}(A, a) &= \{2\}, & \text{cerradura}_\epsilon(\{2\}) &= \{2, 5\} = B, \\ \text{mover}(A, b) &= \{4\}, & \text{cerradura}_\epsilon(\{4\}) &= \{4, 5\} = C. \end{aligned}$$

B y C son estados de aceptación porque ambos contienen al estado 5 del AFN. Desde cualquiera de ellos, leer a o b produce el conjunto vacío $D = \emptyset$, que representa que el AFN ya no tiene un recorrido posible.

Estado del AFD	Con a	Con b	Acepta
$A = \{0, 1, 3\}$	B	C	No
$B = \{2, 5\}$	D	D	Sí
$C = \{4, 5\}$	D	D	Sí
$D = \emptyset$	D	D	No



El AFD ya no necesita elegir entre recorridos. Por ejemplo, con la entrada b sigue $A \xrightarrow{b} C$ y acepta; con ab sigue $A \xrightarrow{a} B \xrightarrow{b} D$ y rechaza. El lenguaje sigue siendo $\{a, b\}$.

Observación 3. Un estado del AFD puede representar varios estados del AFN

Los nombres A , B , C y D sólo simplifican la escritura; cada uno representa un subconjunto. Aunque Q tiene seis estados y $\mathcal{P}(Q)$ contiene 64 candidatos, el recorrido desde A sólo alcanza esos cuatro. Los demás no se incorporan al AFD.

5. Minimización de autómatas

Dos AFD pueden reconocer el mismo lenguaje y tener diferente cantidad de estados. Minimizar consiste en encontrar un AFD equivalente con el menor número de estados.

Definición 6. (Estados distinguibles y equivalentes)

Una cadena x **distingue** dos estados p y q si, al comenzar en ellos y consumir x , exactamente uno de los recorridos termina en aceptación. Dos estados son **equivalentes** si ninguna cadena los distingue.

Si $\hat{\delta}(p, x)$ representa el estado alcanzado desde p después de consumir toda la cadena x , esta relación puede escribirse como

$$p \approx q \iff \forall x \in \Sigma^*, (\hat{\delta}(p, x) \in F \iff \hat{\delta}(q, x) \in F).$$

El algoritmo de minimización por particiones agrupa los estados que todavía no se han distinguido:

1. Divide inicialmente los estados en dos grupos: estados de aceptación y estados que no son de aceptación.
2. Revisa las transiciones de los estados de cada grupo. Si con algún símbolo llegan a grupos distintos, separa esos estados.
3. Repite el refinamiento hasta que ningún grupo pueda dividirse.
4. Sustituye cada grupo final por un solo estado.

Ejemplo resuelto. Minimización del AFD de $a \mid b$

La partición inicial del AFD anterior es

$$\Pi_0 = \{\{B, C\}, \{A, D\}\}.$$

B y C son estados de aceptación y, con a o b , ambos llegan a D . Ninguna continuación los distingue. En cambio, A y D deben separarse: con la cadena a , desde A se llega al estado de aceptación B , mientras que desde D se permanece en un estado que no acepta. La partición estable es

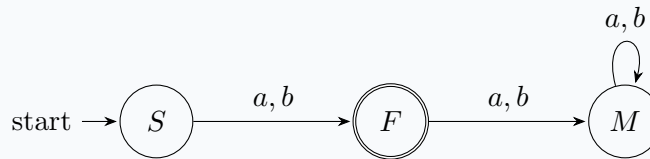
$$\Pi_f = \{\{B, C\}, \{A\}, \{D\}\}.$$

El mismo refinamiento puede cotejarse mediante la **tabla de pares** empleada en las notas de Teoría de la Computación. Primero se marcan los pares formados por un estado de aceptación y otro que no acepta. Después se marca un par pendiente si, con algún símbolo, sus transiciones llegan a un par ya marcado. El procedimiento se repite hasta que no haya cambios.

Par	Resultado	Razón
$\{A, B\}$	Marcado	Sólo B es de aceptación.
$\{A, C\}$	Marcado	Sólo C es de aceptación.
$\{A, D\}$	Marcado por transición	Con a llega a $\{B, D\}$, que ya está marcado.
$\{B, C\}$	Sin marca	Con a o b , ambos estados llegan a D .
$\{B, D\}$	Marcado	Sólo B es de aceptación.
$\{C, D\}$	Marcado	Sólo C es de aceptación.

El único par que permanece sin marca es $\{B, C\}$; por tanto, $B \approx C$. Este resultado coincide con el grupo $\{B, C\}$ obtenido mediante particiones.

Al reunir B y C obtenemos el siguiente AFD mínimo:



S representa $\{A\}$, F representa $\{B, C\}$ y M representa $\{D\}$. La expresión regular, el AFN, el AFD de cuatro estados y este AFD de tres estados reconocen exactamente $\{a, b\}$.

Propiedad. AFD mínimo

Para cada lenguaje regular existe un AFD con un número mínimo de estados, único salvo por los nombres elegidos para esos estados.

Un AFD completo requiere una transición para cada estado y cada símbolo; por ello conserva el estado muerto M . En la implementación de un analizador léxico puede omitirse ese estado y tratar una transición ausente como señal de que ya no es posible extender el lexema. Esta decisión de implementación no cambia el lenguaje reconocido por el autómata completo.

Mini-checkpoint. Coteja el lenguaje después de convertir

Comprueba las entradas ε , a , b , aa y ab en el AFD mínimo.

Salida esperada:

entrada	ε	a	b	aa	ab
resultado	rechaza	acepta	acepta	rechaza	rechaza

Los resultados coinciden con $L(a \mid b) = \{a, b\}$; por tanto, las conversiones del ejemplo conservaron el lenguaje.

6. Aplicación integrada a los tokens de IMP

Regresemos al fragmento inicial y hagamos visible la relación entre entrada, especificación y reconocimiento:

```
while x2 <= 20 do x2 := x2 + 1
```

Cada lexema puede cotejarse con el lenguaje descrito por una expresión regular o por un lexema fijo.

Lexema	Token	Patrón que satisface	Información
while	WHILE	Lexema fijo while.	—
x2	LOC	$\text{letra}^+ \text{digito}^*$.	x2
<=	LE	Lexema fijo <=.	—
20	NUM	digito^+ .	20
do	DO	Lexema fijo do.	—
x2	LOC	$\text{letra}^+ \text{digito}^*$.	x2
:=	ASSIGN	Lexema fijo :=.	—
x2	LOC	$\text{letra}^+ \text{digito}^*$.	x2
+	PLUS	Lexema fijo +.	—
1	NUM	digito^+ .	1

Si representamos los atributos entre paréntesis, la salida conceptual esperada es:

```
WHILE LOC(x2) LE NUM(20) DO  
LOC(x2) ASSIGN LOC(x2) PLUS NUM(1)
```

La transformación puede leerse en tres pasos:

1. **Entrada:** el flujo de caracteres del programa fuente.
2. **Reconocimiento:** los autómatas asociados con los patrones avanzan sobre la entrada y determinan qué cadenas pueden formar lexemas.
3. **Salida:** una secuencia de tokens con los atributos necesarios para las fases posteriores.

Hasta aquí hemos estudiado cada patrón y las conversiones que permiten reconocer su lenguaje. Para obtener el analizador léxico funcional todavía será necesario coordinar todas las reglas, resolver coincidencias como `while` frente a `LOC`, identificar el lexema adecuado dentro del flujo, omitir los espacios permitidos, reportar errores y expresar la especificación en LARK. Esas decisiones corresponden al tema 2.3.

Observación 4. Qué se construye ahora y qué se automatizará después

Las expresiones regulares, los autómatas y las conversiones explican por qué una especificación léxica puede transformarse en un reconocedor. Más adelante, LARK automatizará parte de ese trabajo a partir de las reglas que proporcionemos; no decidirá cuáles son los tokens de IMP ni corregirá una especificación que describa el lenguaje equivocado.

Conclusión

Las expresiones regulares permiten reemplazar descripciones informales por especificaciones precisas de los patrones léxicos. Sus operaciones de unión, concatenación y cerradura construyen lenguajes a partir de símbolos y subexpresiones. En IMP, estas herramientas permiten expresar categorías como `LOC` y `NUM`, además de los patrones fijos de palabras reservadas, operadores y delimitadores.

Los autómatas finitos convierten esa especificación en un procedimiento de reconocimiento. Un AFN puede mantener varias posibilidades y utilizar transiciones ε ; un AFD sigue una sola transición por símbolo. La construcción de Thompson lleva una expresión regular a un AFN, la construcción de subconjuntos produce un AFD equivalente y la minimización reúne estados que no pueden distinguirse. Cada transformación conserva el lenguaje reconocido.

Ya contamos con la base formal para reconocer los patrones del IMP básico. En el tema siguiente coordinaremos las reglas léxicas y construiremos con PYTHON y LARK el analizador que convertirá un programa completo en la secuencia de tokens que recibirá el analizador sintáctico.

Referencias

- [1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson.
- [2] Vilar Torres, J. M. (2010). *Analizador léxico*. Universitat Jaume I.
- [3] Soto Romero, M. (2021). *Autómatas finitos* (Nota de clase 3, Teoría de la Computación). Colegio de Ciencia y Tecnología UACM.
- [4] Soto Romero, M. (2021). *Autómatas finitos con ε -transiciones* (Nota de clase 5, Teoría de la Computación). Colegio de Ciencia y Tecnología UACM.